

Image Classification of Pigmented Skin Lesions for Skin Cancer Detection

Final Project Report

Dona Inayyah

CSC 411 - Interdisciplinary Machine Learning for Data Scientists

San Francisco State University

Fall 2024

Table of Contents

1. **Introduction & Goal**
2. **Dataset Overview**
 - 2.1. Dataset Contents
 - 2.2. Image Data & Metadata
 - 2.3. Tabular Data: hmnist_28_28_RGB.csv
3. **Sample Images**
4. **Exploratory Data Analysis (EDA)**
 - 4.1. Distribution of Target Variable (dx)
 - 4.2. Distribution of Metadata Features
 - 4.3. Dataset Inspection (Missing Values & Duplicates)
5. **Data Preprocessing**
 - 5.1. Preprocessing Image Data for EfficientNetB0
 - 5.2. Preprocessing Tabular Data for Logistic Regression
6. **Modeling Approach & Hyperparameter Tuning**
 - 6.1. Multiclass Logistic Regression
 - 6.2. EfficientNetB0 - Transfer Learning
7. **Model Evaluation & Results**
 - 7.1. Evaluation Metrics
 - 7.2. Results
 - 7.3. Test Score Analyses
8. **Roadblocks & Challenges**
9. **Learning Outcomes & Future Direction**
 - 9.1. Key Learning Outcomes
 - 9.2. Future Improvements
10. **Individual Contribution**
11. **Colab Notebook**
12. **References**

1. Introduction & Goal

Skin lesions are areas of skin that visibly differ from the surrounding tissue, ranging from benign growths to malignant conditions (Oakley, 2015). Skin cancers, arising from these lesions, are categorized into basal cell carcinoma, squamous cell carcinoma, and melanoma. According to the Skin Cancer Foundation (2023), non-melanoma skin cancers (basal and squamous cell carcinomas) are highly treatable if detected early, while melanoma, though less common, is significantly more dangerous due to its higher likelihood of spreading.

Our project aims to develop machine learning models to classify medical images of pigmented skin lesions into the following seven categories:

- **Actinic keratoses:** Precancerous lesions that may develop into squamous cell carcinoma.
- **Basal cell carcinoma:** A common, treatable type of skin cancer.
- **Melanoma:** A dangerous skin cancer with high metastatic potential.
- **Benign keratosis-like lesions:** Non-cancerous but sometimes mimic melanoma.
- **Dermatofibroma:** Benign fibrous growths.
- **Vascular lesions:** Benign abnormalities of blood vessels.
- **Melanocytic nevi:** Common moles, generally harmless.

Accurately identifying these lesions is critical for early detection and timely intervention, leading to better treatment outcomes and a reduced burden of skin cancer.

2. Dataset Overview

Source: [Skin Cancer MNIST: HAM10000](#)

The Skin Cancer MNIST: HAM10000 dataset, also known as HAM10000 (“Human Against Machine with 10000 training images”), was sourced from Kaggle. It contains 10,015 dermatoscopic images collected from diverse populations, representing key diagnostic categories for pigmented lesions. The dataset covers seven classes of lesions:

- `akiec`: Actinic keratoses and intraepithelial carcinoma / Bowen's disease
- `bcc`: Basal cell carcinoma
- `bkl`: Benign keratosis-like lesions
- `df`: Dermatofibroma
- `mel`: Melanoma
- `nv`: Melanocytic nevi
- `vasc`: Vascular lesion

2.1. Dataset Contents

The dataset comprises both **image data** and **metadata**, structured as follows:

1. **Image Directories:**

- `ham10000_images_part_1`
- `ham10000_images_part_2`

2. **CSV Files:**

- **`HAM10000_metadata.csv`**
- `hmnist_8_8_L.csv`
- `hmnist_8_8_RGB.csv`
- `hmnist_28_28_L.csv`
- **`hmnist_28_28_RGB.csv`**

The ones in **bold** are the datasets we used for our project.

2.2. Image Data & Metadata

The **target variable** for the dataset is `dx`, which represents the diagnostic category of each image. The **primary input** for the transfer learning models comprises the **pixel values** extracted from the image directories. These pixel values serve as the foundational features for training the EfficientNetB0 models.

Features in Metadata:

- `lesion_id`: Unique identifier for each lesion case.
- `image_id`: Unique identifier for each image.
- `dx_type`: Diagnostic method used to confirm each lesion's diagnosis.
- `age`: Patient's age at the time of biopsy.
- `sex`: Patient's gender.
- `localization`: Body area where the lesion is located.

While the metadata features were not used directly in model training, they provided valuable context about the dataset's population.

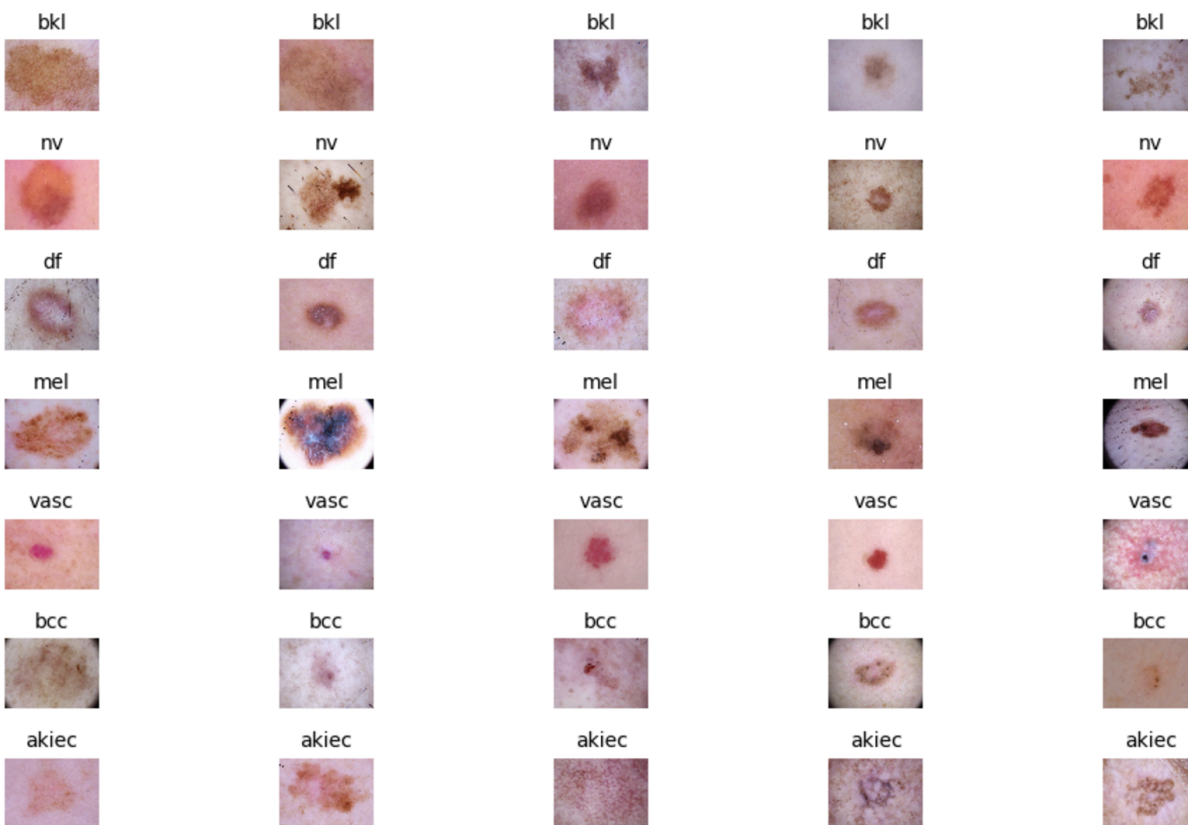
2.3. Tabular Data: `hmnist_28_28_RGB.csv`

This file contains flattened image data in a tabular format, offering an alternative structure to run a logistic regression model.

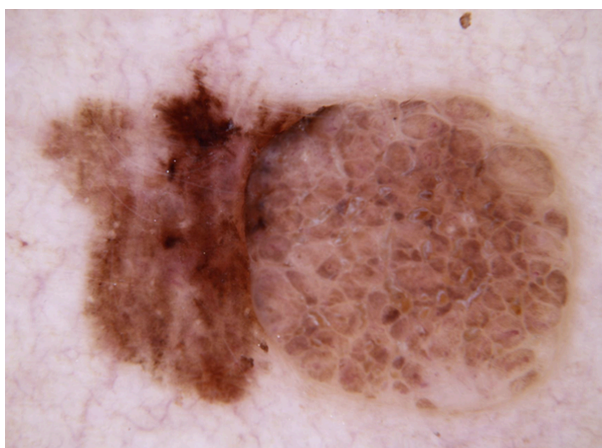
The **features** consist of pixel values, represented as individual columns labeled as `pixel0000`, `pixel0001`, and so on, corresponding to the pixels in a 28×28×3 RGB image. In total, the dataset includes 2352 pixel columns and one additional column, `label`, which serves as the **target variable**. Overall, the dataset comprises 10,015 rows and 2,353 columns, providing a compact representation of image data for tabular-based machine learning methods.

3. Sample Images

Images from each Class



Original Image from Directory



Resolution: 600 x 450

Flattened Image from CSV

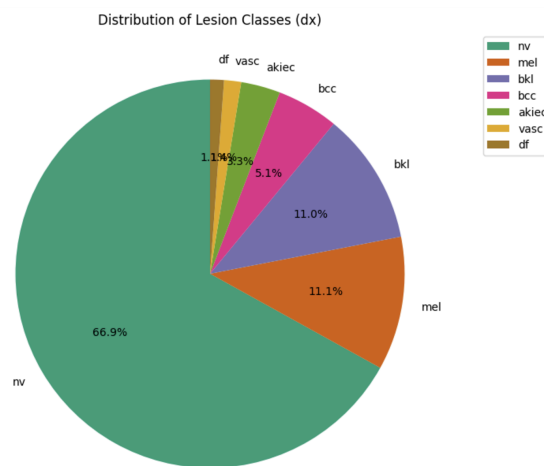


Resolution: 28 x 28

4. Exploratory Data Analysis (EDA)

Before preprocessing, we conducted Exploratory Data Analysis to investigate the dataset and gain insights into its structure. Our objectives were to analyze the distributions of the target variable and metadata features, identify class imbalances, and check for missing values or duplicates. Below are the key findings:

4.1. Distribution of Target Variable - dx

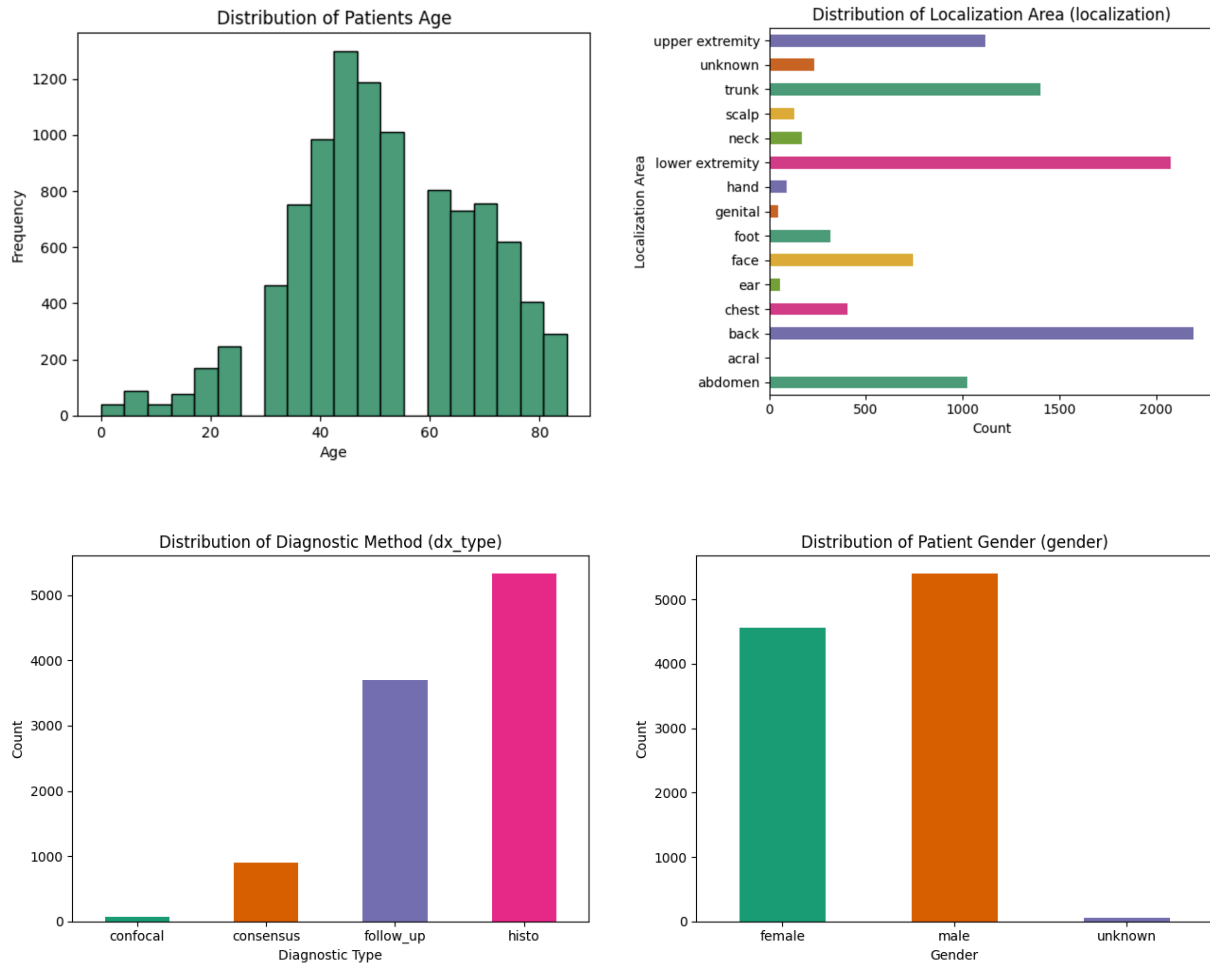


From the pie chart, we observe significant class imbalance in the dataset:

- Melanocytic nevi (nv) dominates the dataset, accounting for 66.9% of the total images.
- Melanoma (mel) and benign keratosis-like lesions (bkl) are the next largest classes, comprising 11.1% and 11.0% of the data, respectively.
- The remaining 11.2% is distributed among the four smaller classes:
 - Actinic keratoses (akiec)
 - Basal cell carcinoma (bcc)
 - Vascular lesions (vasc)
 - Dermatofibroma (df), making up only 1.1% of the dataset.

This imbalance highlights the need for techniques such as data augmentation and resampling to ensure balanced representation across all classes during model training. Additionally, while the majority of patients had benign skin lesions, a significant portion of individuals were diagnosed with melanoma, the most serious and potentially life-threatening type of skin cancer.

4.2. Distribution of Metadata Features



I. Age Distribution

The age distribution of patients follows a near-normal distribution, with the 40-50 age range being the most frequent. This suggests that middle-aged individuals are most represented in the HAM dataset.

II. Localization of Lesions

The localization area distribution shows that the back has the highest number of lesions, with over 2000 occurrences. Other frequently observed areas include the:

- Lower extremity
- Trunk

- Abdomen
- Upper extremity, each with counts exceeding 1000.

In contrast, other body areas have fewer than 750 occurrences.

III. Diagnostic Method Distribution

The diagnostic method bar chart indicates that histopathology is the most common method for confirming diagnoses, accounting for approximately 5000 cases. In contrast, confocal microscopy is the least used method, with values close to zero.

IV. Gender Distribution

The gender distribution shows a slightly higher number of male patients compared to females:

- Male patients: ~5500
- Female patients: ~4500
- A small number (<10) of patients are recorded with an unknown gender.

4.3. Dataset Inspection - Missing Values and Duplicates

Missing Values:

The dataset is mostly complete, with 57 missing values in the age column. Since age is not currently used as a feature for our models, we did not address this missing data. However, if age is incorporated in future analyses, we will consider appropriate imputation techniques (e.g., median or mean imputation).

Duplicate Entries:

We verified the dataset for duplicates using the `image_id` column and confirmed that each image has a unique identifier. No duplicates were found, ensuring data integrity.

5. Data Pre-Processing

Effective preprocessing is critical for improving model performance, especially in skin cancer datasets where variations in lighting, resolution, and backgrounds are common. Since we used two datasets—image data for the EfficientNetB0 model and tabular data for logistic regression—each required distinct preprocessing steps.

5.1 Preprocessing Image Data for EfficientNetB0

I. Train-Test Split

To evaluate model performance and prevent overfitting, we split the data into training (70%), validation (10%), and test (20%) sets. We used `train_test_split` from `sklearn` for this process. The final split sizes were:

- **Train:** 7210 images
- **Validation:** 802 images
- **Test:** 2003 images

The training set allows the model to learn features, the validation set helps fine-tune hyperparameters, and the test set evaluates the model's generalization ability.

II. Mapping Image files to Metadata

The image dataset was provided in two separate directories. We combined these directories and matched the images to the corresponding metadata in `HAM10000_metadata.csv` using their `image_id`. This ensured each image was accurately associated with its corresponding diagnostic label (`dx`).

III. Data Augmentation to Handle Imbalanced Classes

As identified in the EDA section, the dataset exhibited severe class imbalance:

- bcc: 376 | df: 78 | bkl: 780 | mel: 789 | nv: 4838 | vasc: 114 | akiec: 235

To address this, we employed data augmentation for minority classes using TensorFlow's `ImageDataGenerator`. Techniques such as flipping, rotation, zooming, and noise addition were applied to generate synthetic images while ensuring no duplicates.

To balance the classes:

- Minority classes were upsampled to reach a target size of 850 images per class.
- The majority class (nv) was downsampled to the same size.

Initially, we considered setting the target size to the median class count (~ 300), but it was insufficient to maintain the 70/20/10 train-validation-test split. Instead, we set a target size of 850 images per class, balancing class representation and dataset size. The final split was: Train: 5864, Validation: 802, and Test: 2003 images.

IV. Normalization & Resizing

To prepare the images for EfficientNetB0:

- **Normalization:** Pixel values were rescaled to the range $[0, 1]$ by dividing each pixel by 255 using `ImageDataGenerator`. This step improves model stability and convergence during training.
- **Resizing:** EfficientNetB0 requires input images of size 224×224 pixels. Since the original images were 600×450 , we resized them accordingly.

5.2. Preprocessing for Logistic Regression

I. Train-Test Split

We applied the same **70/10/20 split** strategy using `train_test_split` from `sklearn`.

II. Resampling Imbalanced Classes

Class imbalance was addressed using the `resample` function from `sklearn.utils`. Using the same strategy, minority classes were upsampled, and the majority class was downsampled to achieve 850 samples per class.

III. Normalization

The pixel values were normalized using the `MinMaxScaler`, which scaled all values to the range $[0, 1]$. This step ensures uniformity in the data and improves the performance of the logistic regression model.

6. Modeling Approach & Hyperparameter Tuning

For our image classification task, we implemented two distinct models, each serving a specific purpose in evaluating and improving classification performance: Multiclass Logistic Regression and EfficientNetB0 (Transfer Learning).

6.1. Multiclass Logistic Regression

Our first model was multiclass logistic regression, which served as a baseline model. Logistic regression is a simple and interpretable model that directly maps input features to output probabilities without relying on complex architectures. By providing a minimal-effort benchmark, it allowed us to assess the improvements offered by more advanced models.

Model Architecture:

- **Input Type:** The model was trained on flattened image data from the `hmnist_28_28_RGB.csv` file. Each image was represented as a 1D array of 2352 pixel values ($28 \times 28 \times 3$ for RGB images).
- **Loss Function:** The loss function used was categorical cross-entropy, which is standard for multi-class classification tasks.
- **Output Layer:** A single-layer logistic regression model that outputs probabilities for each of the 7 diagnostic categories using a softmax activation function.
- **Input Size:** 2352 features (flattened pixel values).
- **Target Variable:** A single integer value representing the class labels (0–6).

The decision to use RGB images instead of grayscale was made to preserve crucial color-based information. However, due to the dataset's low resolution (28×28 pixels), we anticipated a loss of spatial relationships and lower performance.

To optimize the model:

- **Hyperparameter Tuning:** We attempted automating the tuning process using `GridSearchCV` and for loops. However, the dataset's size posed computational challenges, leading us to perform manual tuning through trial and error.

- **Final Hyperparameters:** After experimentation, the chosen hyperparameters were:
 - Solver: 'lbfgs'
 - Regularization strength (C): 1
 - Maximum Iterations (max_iter): 1000

This baseline model, while limited in its ability to capture complex spatial relationships inherent in image data, provided a useful lower bound for model performance.

6.2. EfficientNetB0 - Transfer Learning

Our second approach employed EfficientNetB0, a pre-trained deep learning model leveraging transfer learning. EfficientNet was chosen for its ability to strike a balance between model complexity and computational efficiency, making it ideal for moderately sized datasets like HAM10000.

Model Architecture:

- **Input Type:** Images resized to 224×224×3 pixels to match EfficientNetB0's input requirements.
- **Base Model:** EfficientNetB0 pre-trained on ImageNet with weights frozen during initial training.
- **Custom Layers Added:**
 - **Global Average Pooling Layer:** Reduces the output of the EfficientNet feature maps into a single vector per image.
 - **Fully Connected Dense Layer:** 128 neurons with ReLU activation for learning dataset-specific features.
 - **Output Layer:** 7 neurons with softmax activation to predict probabilities for the 7 classes.
- **Loss Function:** Categorical Cross-Entropy Loss.
- **Optimizer:** Adam Optimizer with an initial learning rate of 0.001 (lowered to 1e-5 during fine-tuning).

I. First Iteration

Model: "sequential_1"

Layer (type)	Output Shape	Param #
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None, 1280)	0
dense_5 (Dense)	(None, 128)	163,968
dense_6 (Dense)	(None, 7)	903

Total params: 4,214,442 (16.08 MB)
 Trainable params: 164,871 (644.03 KB)
 Non-trainable params: 4,049,571 (15.45 MB)

In the first iteration, EfficientNetB0 was used as a feature extractor with its pre-trained layers frozen. Only the custom classification layers were trained. We then:

- Unfroze the EfficientNetB0 layers and fine-tuned the entire model with a lower learning rate.
- Used Early Stopping to monitor validation loss, ensuring training halted once performance plateaued.

II. Second Iteration

Model: "sequential_4"

Layer (type)	Output Shape	Param #
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d_7 (GlobalAveragePooling2D)	(None, 1280)	0
dense_12 (Dense)	(None, 128)	163,968
dropout_1 (Dropout)	(None, 128)	0
dense_13 (Dense)	(None, 7)	903

Total params: 4,214,442 (16.08 MB)
 Trainable params: 164,871 (644.03 KB)
 Non-trainable params: 4,049,571 (15.45 MB)

Although the first iteration trained successfully, we identified slight signs of overfitting. To address this, we:

- Added L2 regularization to the dense layer to penalize overly complex weights.
- Implemented a Dropout layer (50% rate) to randomly deactivate neurons during training, improving generalization.
- Increased the maximum number of epochs to 50 while still using Early Stopping.

7. Model Evaluation & Results

To assess the performance of our models, we used a set of evaluation metrics suitable for multi-class classification tasks. Given the medical context of our project, we prioritized recall as the primary metric due to its importance in identifying as many true positive cases as possible, minimizing the risk of false negatives

7.1. Evaluation Overview:

I. Confusion Matrix

The confusion matrix is a 7×7 matrix representing the counts of true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN) for each class. It provides a detailed breakdown of the model's predictions for all 7 diagnostic categories.

II. **Accuracy** is the proportion of correctly classified instances over the total instances.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

III. **Precision** measures how many of the positive predictions made by the model are actually correct.

$$Precision = \frac{TP}{TP + FP}$$

IV. **Recall (Sensitivity)** indicates how well the model identifies all true positive cases. It is particularly important in medical applications where false negatives are costly.

$$Recall = \frac{TP}{TP + FN}$$

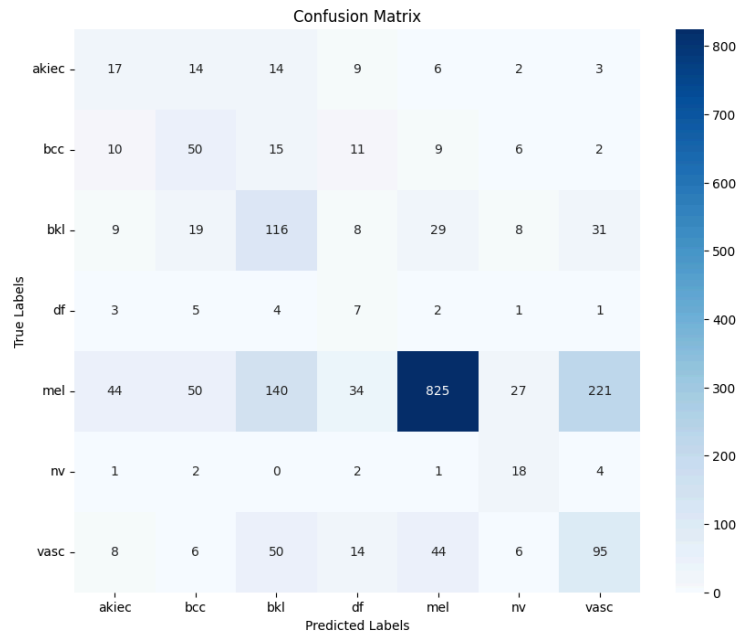
V. **F1-Score** is the harmonic mean of precision and recall, balancing these two metrics. It is useful when dealing with class imbalances.

$$F1\ Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

VI. **Log Loss (Categorical Cross Entropy)** evaluates how well the predicted probabilities align with the actual class labels. Lower values indicate better performance.

7.2. Results

I. Multiclass Logistic Regression - Base Model

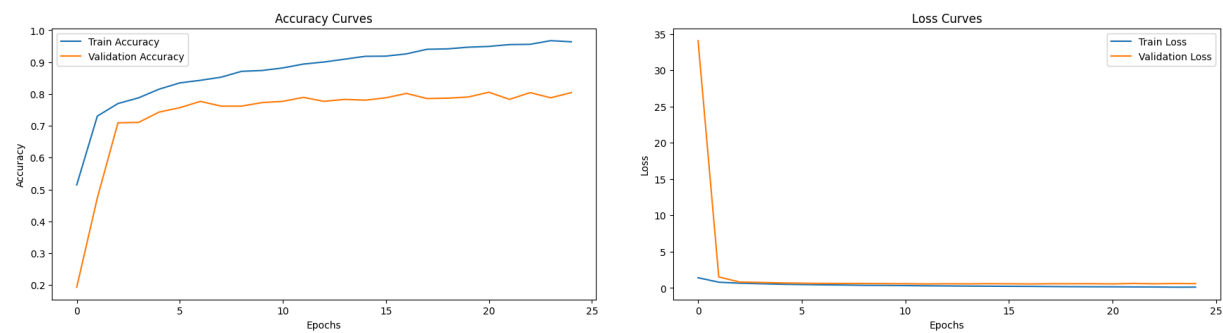


Metric	Test
Accuracy	0.563155
Recall	0.563155
F1-Score	0.605907
Precision	0.698448
Log Loss	1.345552

Observations:

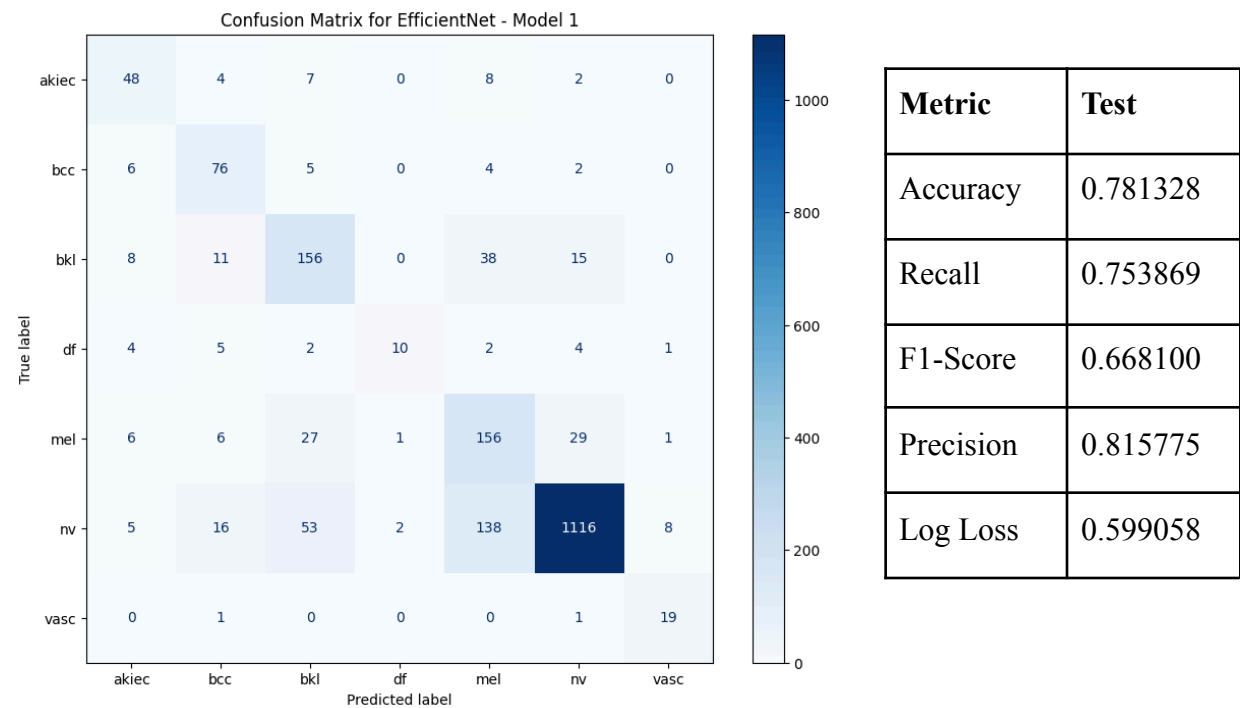
- Performance was poor, as expected, due to the model's inability to capture spatial information inherent in images.

II. EfficientNetB0 - Iteration 1

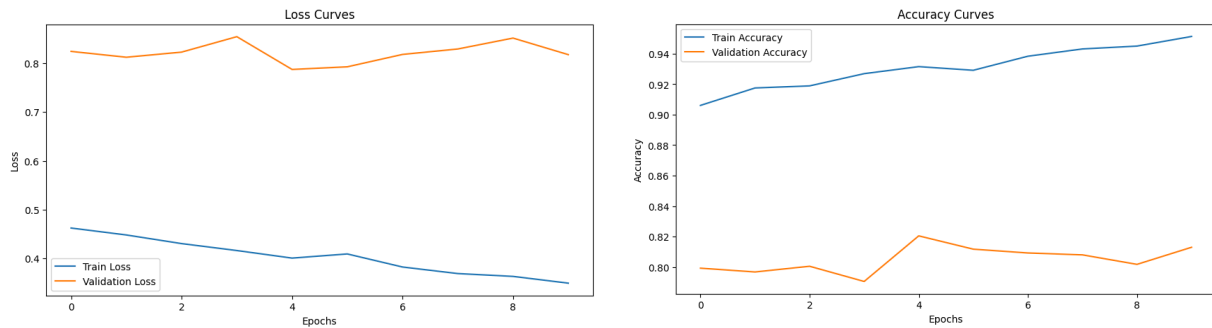


Observations:

The training loss decreased steadily over the epochs, indicating that the model was effectively learning patterns from the data. Validation accuracy stabilized around 80–85%, suggesting that the model was performing well on unseen data. Additionally, no significant signs of overfitting were observed as the validation loss remained closely aligned with the training loss, which indicates that the model wasn't memorizing the data. Finally, the model trained over 25 epochs which was the maximum number set for early stopping.

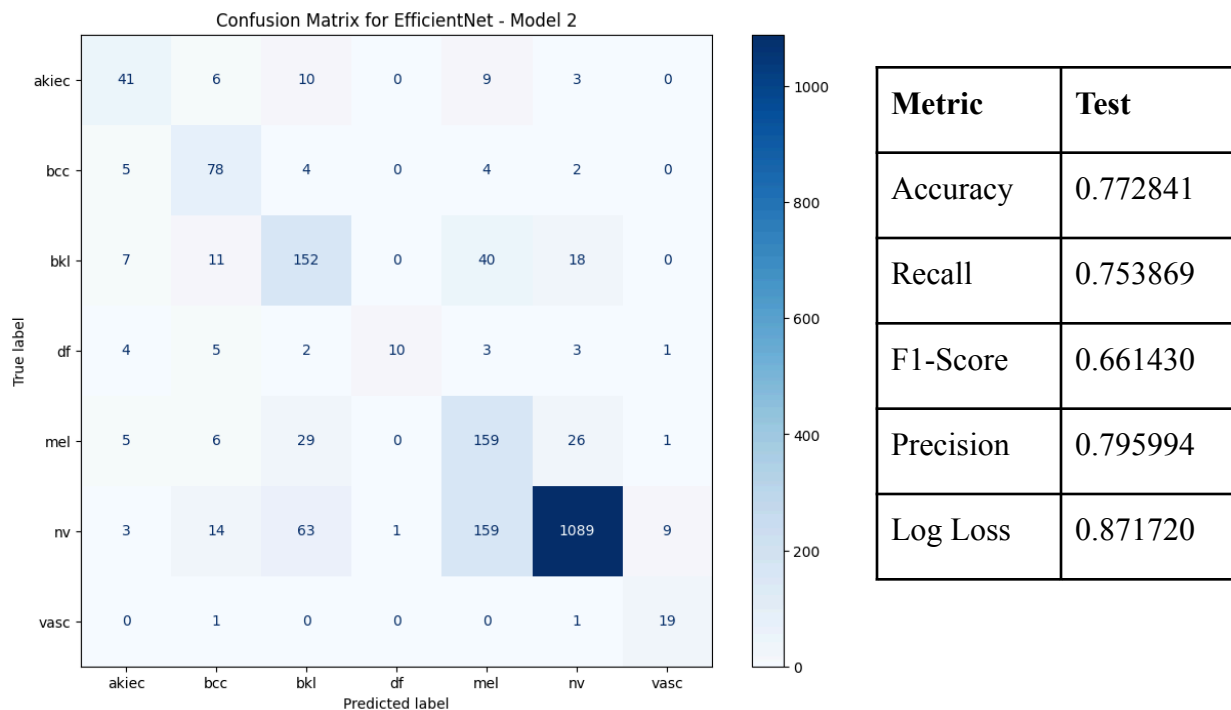


III. EfficientNetB0 - Iteration 2



Observations:

Despite these enhancements, the model's performance in this iteration was much worse than in the first, with early stopping halting at 10 epochs due to a lack of improvement in validation loss. This suggests that the added regularization and dropout, while theoretically beneficial, may have overly constrained the model in this case.



7.3. Test Score Analyses

I. Multiclass Logistic Regression

The logistic regression model, used as a baseline, achieved an accuracy of 56.3%. While its precision was relatively high at 69.8%, recall remained low at 56.3%, reflecting the model's struggle to generalize on imbalanced image data. The high log loss value of 1.346 further indicated poor confidence in its predicted probabilities.

II. EfficientNetB0 - Iteration 1

The first iteration of EfficientNetB0 significantly outperformed logistic regression, achieving an accuracy of 78.1% and a recall of 75.4%. The F1-score improved to 66.8%, and the precision was strong at 81.6%, indicating reliable classification across most classes. The log loss value of 0.599 demonstrated better confidence in predictions. The model effectively leveraged transfer learning to extract meaningful spatial features, making it well-suited for the dataset.

III. EfficientNetB0 - Iteration 2

In the second iteration, L2 regularization and a dropout layer were added to address potential overfitting. While recall improved slightly to 76.1%, the F1-score dropped marginally to 66.6%, and precision decreased to 80.2%. The log loss increased to 0.854, suggesting that these modifications may have overly constrained the model. Early stopping halted training after 10 epochs, indicating limited improvement during fine-tuning.

IV. Conclusion

EfficientNetB0 - Iteration 1 delivered the best overall performance, achieving the highest accuracy, recall, and precision, along with the lowest log loss. The second iteration, while incorporating additional regularization techniques, showed limited improvement, which highlighted the need for careful fine-tuning to avoid over-constraining the model. In contrast, the logistic regression model, as expected, struggled to capture the spatial relationships crucial for accurate image classification, particularly for minority classes. Flattening the pixel values resulted in significant information loss, and the low resolution (28×28 pixels) further hindered its ability to generalize, leading to overall poor performance.

8. Roadblocks & Challenges

I. File Management and OS Issues

One of the initial challenges was implementing a file-moving system to organize images and match them with their corresponding `image_id` from the metadata. This process was time-consuming because it involved working with new concepts like file directory handling and ensuring the correct association between image files and metadata.

- **Solution:** We resolved this by iteratively debugging and testing the file-moving system, which ultimately ensured that every image was accurately mapped to its label in the metadata.

II. Model Training Bottlenecks

- **Logistic Regression:** Hyperparameter tuning was challenging due to the large size of the dataset. Using automated methods like GridSearchCV proved inefficient because it significantly increased runtime and consumed extensive computational resources.
 - **Solution:** We opted for manual tuning through trial and error to identify the optimal hyperparameters, balancing runtime efficiency and performance.
- **EfficientNetB0:** Despite leveraging a GPU, training the transfer learning model was computationally expensive and time-intensive due to the complexity of the EfficientNet architecture. This remained an unresolved challenge.

III. Memory Constraints

Repeated runtime disconnections and memory limitations were a significant hurdle during training, especially when handling augmented datasets. This led to lost progress and required reloading data multiple times.

- **Solution:** We trained one deep learning model at a time. However, this limited the extent of experiments and iterations we could perform.

9. Learning Outcomes & Future Direction

9.1 Key Learning Outcomes

A key takeaway from this project was learning to work with transfer learning models like EfficientNetB0. I discovered the importance of freezing and unfreezing layers during fine-tuning; freezing the base layers initially allowed the model to focus on training the custom layers, while unfreezing them later enabled fine-tuning for improved performance. I also learned the importance of GPU's, as leveraging it proved essential for efficiently processing large datasets and handling resource-intensive tasks. Finally, I gained significant experience in debugging and problem-solving when working with real-world datasets. Issues such as file organization and operating system errors required me to trace problems, test solutions iteratively, and learn the nuances of directory handling in Python.

9.2. Future Improvements

I. For Logistic Regression:

To enhance the performance of the logistic regression model, I would start by identifying key features using feature importance plots. This step would allow us to reduce the number of input features, simplifying the model and potentially improving its accuracy and efficiency. Reducing feature dimensionality could also lower runtime and make the model more effective, particularly for large datasets.

II. For Deep Learning:

Given that the second iteration of the EfficientNet model underperformed compared to the first, a future improvement would be to gradually unfreeze layers during fine-tuning instead of unfreezing the entire model at once. This approach would allow the model to retain the pretrained knowledge while adapting more effectively to the specific dataset.

Additionally, exploring alternative transfer learning architectures like VGG16 or ResNet50 could provide valuable insights into their relative performance. Building and training a custom Convolutional Neural Network (CNN) tailored to our dataset would also be a promising direction, offering an opportunity to optimize the architecture specifically for this task.

10. Individual Contribution

My individual contributions to this project include:

- **Dataset Selection & Planning:** While all of us searched for datasets, we ended up using the one I found because it best fit our goals. I also planned the project workflow and divided tasks fairly so everyone contributed while learning something new.
- **Technical Work:** I led the coding part of the project, setting up the datasets, organizing images into directories, and handling class imbalances through data augmentation. I also trained and evaluated both models.
- **Editing & Collaboration:** I cleaned up and refined the code from my teammates to make it more consistent. While my teammates focused more on the slides, I helped edit them to improve clarity and flow.
- **Proposal Writing:** I wrote the *Introduction & Goal*, *Data Preprocessing*, and *Modeling Approach* sections, and made sure the proposal came together as a cohesive document.

Overall, I am very happy with how our team collaborated throughout the project. Everyone was proactive and took initiative, which allowed us to stay on track, meet deadlines, and be well-prepared. We effectively played to our individual strengths, ensuring that each team member made meaningful contributions. I believe my contributions were particularly impactful in the coding aspects and in helping my teammates understand the rationale behind the project pipeline and decisions, which ultimately strengthened our overall workflow and execution.

11. Final Project Notebook

The complete project notebook containing the code and results can be found [here](#).

References

Early detection. The Skin Cancer Foundation. (2023, February 27).

<https://www.skincancer.org/early-detection/>

Oakley, A. (2015, October). *Pigmented skin lesions*. DermNet®.

<https://dermnetnz.org/topics/pigmented-skin-lesions>